



Character tables

Fifty years ago, biologists constructed phylogenies not from DNA and protein sequences but from anatomical or physiological properties called **characters**. For example, when analyzing invertebrate evolution, one commonly used character is the presence or absence of wings, while another is the number of legs (varying from 0 to over 300 in some centipedes). These two characters result in the 3×2 **character table** shown in the figure below for three species.



	wings	legs
winged stick insect	Yes	6
wingless stick insect	No	6
giant centipede	No	42

Figure: (Top panel) Winged (left) and wingless (middle) stick insects, each having six legs, and the giant centipede (right), which has 42 legs. (Bottom panel) A 3×2 character table describes two characters (wings and legs) in these three invertebrates. Image courtesies: Drägüs, Wikimedia Commons user (winged stick insect), L. Shyamal (wingless stick insect), Katka Nemčoková (giant centipede)



In general, every row in an $n \times m$ character table holds one of m characters corresponding to one of n existing species. Our goal, roughly stated, is to use a character table to construct an evolutionary tree in which present-day species with similar values for the same character occur near each other. We would also like to assign m character values to each internal node in the tree in order to best explain the characters of ancestral species.

STOP and Think: Can you convert the above vague description into a well-formulated computational problem that models tree reconstruction based on character tables?





From anatomical to genetic characters

In 1965, Emile Zuckerkandl and Linus Pauling published “Molecules as documents of evolutionary history”, arguing that DNA sequences offer a much more informative source of data than anatomical and physiological characters. The idea may seem obvious today, especially after we have spent half of a chapter constructing evolutionary trees from distance matrices generated from DNA sequences. However, Zuckerkandl and Pauling’s proposal was initially met with skepticism by many biologists, who felt that DNA analysis could not offer the same power as anatomical comparison, say of inferring how birds had evolved to fit their environment based on studying their distinct beak shapes. A now-famous argument was initiated when Zuckerkandl and Pauling found that the amino acid sequence of human beta-hemoglobin is very similar to that of gorillas, prompting Zuckerkandl to write in 1963:

From the point of view of hemoglobin structure, it appears that gorilla is just an abnormal human.

Yet the surprising similarity between hemoglobin proteins in various primates flew in the face of clear physical differences between primates. As a result, the leading evolutionary biologist Gaylord Simpson immediately responded to Zuckerkandl:

...that is of course nonsense. What the comparison really indicates is that hemoglobin is a bad choice and has nothing to tell us about attributes, or indeed tells us a lie.



Despite such vehement initial criticisms, genetic analysis had become the dominant technique in evolutionary studies by the 1970s. Indeed, the analysis of DNA sequences answered evolutionary questions that previous analysis of anatomical characters had failed to resolve. Early examples include the classification of the giant panda (see [DETOUR: Giant Panda: Bear or Raccoon?](http://stepic.org/lesson/Detour-Giant-Panda-Bear-or-Raccoon-10344/) (<http://stepic.org/lesson/Detour-Giant-Panda-Bear-or-Raccoon-10344/>)) and the identification of human origins (see [DETOUR: Where Did Humans Come From?](http://stepic.org/lesson/Detour-Where-Did-Humans-Come-From-10346/) (<http://stepic.org/lesson/Detour-Where-Did-Humans-Come-From-10346/>)). As a result, most evolutionary experts had no choice but to adapt their views and become experts on molecular evolution.

Ironically, modern studies often view an $n \times m$ multiple alignment as a $n \times m$ character table, with each column representing a character of its own. Our goal is to construct a tree whose leaves correspond to the rows of this alignment and whose internal nodes correspond to ancestral sequences in accordance with the most parsimonious evolutionary scenario. Before describing how to achieve this goal, we will describe one example of how this algorithmic framework solved a longstanding puzzle in insect evolution.





How many times has evolution invented insect wings?

Wings provided a revolutionary adaptation for insects, allowing them to escape predators and disperse into new territories, thus leading to many new insect species. Yet despite the evolutionary advantages provided by wings, some insects are apparently better equipped for survival without them. In fact, nearly all winged species are related to many wingless counterparts belonging to the same genus, and some entire orders of insects are wingless, including fleas and lice.

The acquisition of wings would seem to pose an evolutionary challenge because complex physiological interactions are required to accommodate flight. As a result, we would be led to believe that wings evolved only once in insects. This argument parallels **Dollo's principle of irreversibility**, a hypothesis proposed by 19th Century paleontologist Louis Dollo. According to this principle, when a species loses a complex organ, such as wings, the organ will not reappear in exactly the same form in the species's descendants.

STOP and Think: What do you think about this argument as it pertains to insect wings?



Until recently, biologists followed Dollo's principle with respect to insect wings, believing that re-evolution of wings was essentially impossible because unused flight genes in wingless insects would be free to accumulate mutations, eventually eroding into non-functional pseudogenes. However, in 2003, Michael Whiting studied various winged and wingless stick insects from around the world and refuted this argument. He sequenced an approximately 2,000 nucleotide-long segment (the 18S ribosomal RNA gene) from these stick insects and constructed an evolutionary tree based on these sequences. From this phylogeny, he inferred that wings were re-invented at least four times and lost at least three times during stick insect evolution (figure below).

STOP and Think: Is it possible to infer an evolutionary scenario from the tree below that reinvents wings fewer than four times?

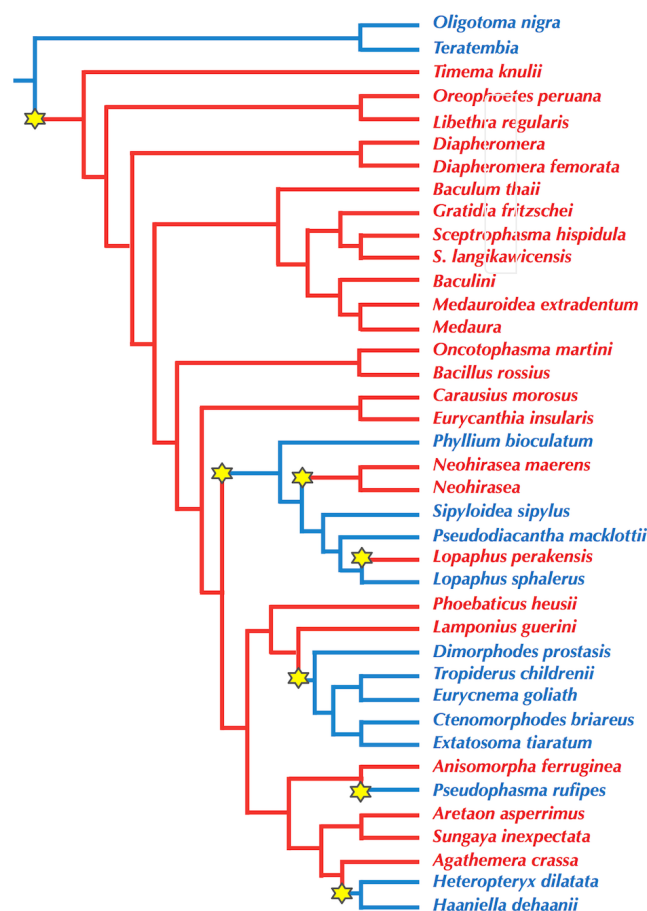


Figure: Evolutionary tree of winged (blue) and wingless (red) stick insects constructed from 18S ribosomal RNA genes. Transitions between winged and wingless species are shown by yellow stars. 18S ribosomal RNAs are slow-evolving, making them ideal for reconstructing ancient divergences.



Whiting's work indicates the inherent complications of trying to infer evolutionary trees from anatomical characters. If you were asked to design a character-based phylogeny algorithm for the collection of all stick insects, then one of the first steps you would probably take would be to cluster all the winged insects on the opposite side of the tree from all of the wingless insects. However, the re-evolution of insect wings means that such an approach is flawed. Anatomical characters have even less power when we move into the microscopic world: just imagine trying to construct a phylogeny from direct observation of coronaviruses!





We will label each leaf of a tree by a row of a multiple alignment, and will attempt to infer strings labeling internal nodes and corresponding to candidate ancestral sequences. However, we will need to develop a scoring function in order to quantify how well such a labeled tree fits the given multiple alignment. In what follows, we assume for simplicity that the multiple alignment contains only substitutions and no indels. In practice, researchers may start from a multiple alignment containing indels and then remove all columns containing indels.





An intuitive score of an evolutionary tree is the total number of mutations required to explain the strings at all nodes of the tree. Given a tree T with every node labeled by a string of length m , we will therefore set the length of edge (v, w) equal to the number of substitutions (Hamming distance) between the strings labeling v and w . The **parsimony score** of T is the sum of the lengths of its edges (see the figure below).

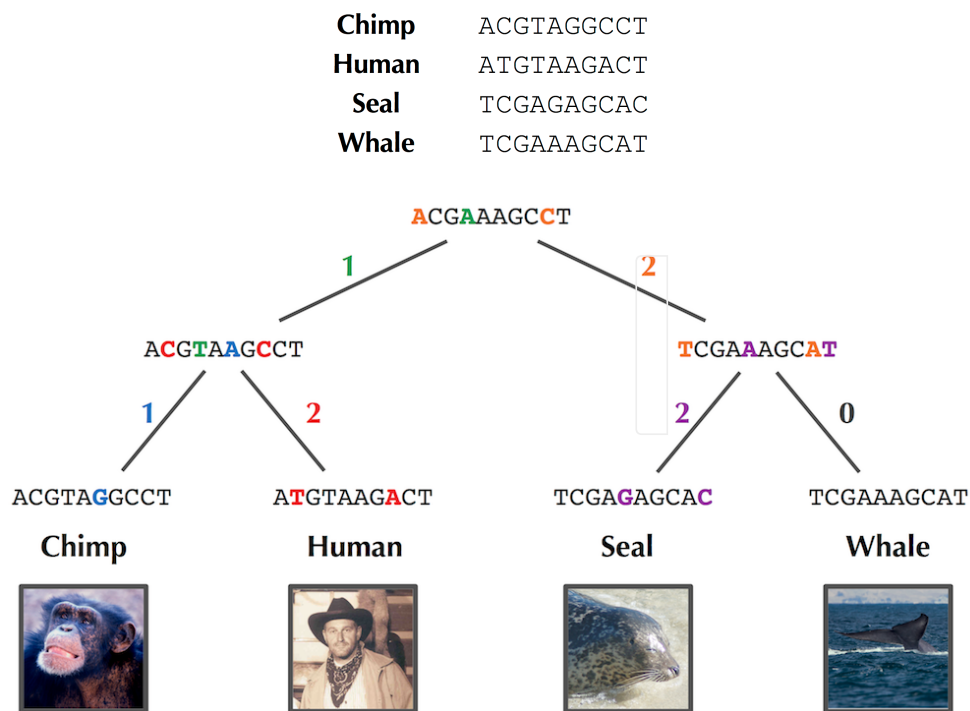


Figure: (Top) The toy mammal multiple alignment we started working with earlier in the chapter. (Bottom) An evolutionary tree with parsimony score 8 whose leaves are the DNA strings in this multiple alignment. Colored letters indicate mismatches in strings connected by an edge.



We will first assume that we are given the structure of a rooted binary tree in advance, in which case we only need to assign strings to the internal nodes to minimize the parsimony score.

Small Parsimony Problem. *Find the most parsimonious labeling of the internal nodes of a rooted tree.*

Input: A rooted binary tree with each leaf labeled by a string of length m .

Output: A labeling of all other nodes of the tree by strings of length m that minimizes the parsimony score of the tree.

Given a tree T for which every node v is labeled by a string of length m , we define labeled trees $T_1, \dots, T_m$, where T_i is the tree T with every node labeled by the i -th symbol of v . Since the parsimony score of T is the sum of parsimony scores of the trees $T_1, \dots, T_m$, the Small Parsimony Problem can be solved independently for each column of the alignment. In other words, given a tree with leaves labeled by strings, we can assign a symbol to the k -th position of each string labeling an internal node without paying attention to the symbols at any other positions. This observation lets us assume that every leaf is labeled by a single symbol rather than by a string. Thus, the weight of an edge connecting two nodes should be either 0 or 1, depending on whether these nodes are labeled by the same symbol or different symbols; given symbols i and j , we define $\delta_{ij} = 0$ if $i = j$ and $\delta_{ij} = 1$ if $i \neq j$.



We will describe a dynamic programming algorithm, called **SmallParsimony**, for solving the “single character” version of the Small Parsimony Problem that was proposed by David Sankoff in 1975. Recall that a rooted tree T can be viewed as a directed tree with all of its edges directed away from the root toward the leaves. Thus, every node v in T defines a subtree T_v formed by the nodes “beneath” v and consisting of all the nodes that can be reached by moving down from v (see figure below).

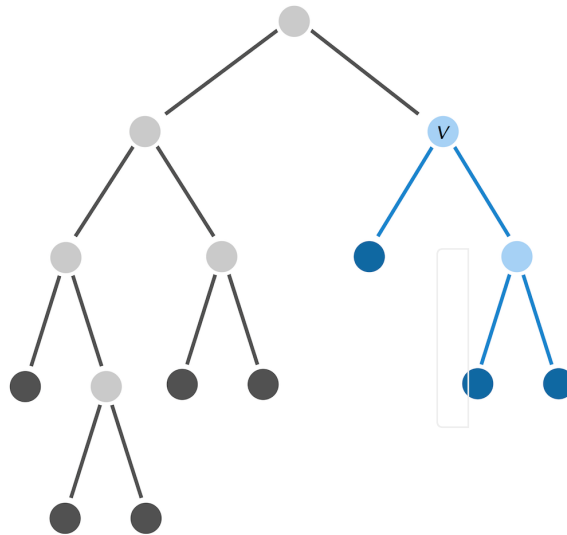


Figure: The (blue) subtree T_v of a node v within a larger rooted binary tree T .



Let k be a symbol in a given alphabet and v be a node in a tree T . Define $s_k(v)$ as the minimum parsimony score of the subtree T_v over all possible labelings of the nodes of T_v such that v is labeled by k . The initial conditions for **SmallParsimony** must assign scores to leaves. If leaf v is labeled by the symbol k , then the only character we are allowed to assign to this leaf is k . Therefore, $s_k(v) = 0$ if leaf v is labeled by symbol k , and $s_k(v) = \infty$ otherwise (see figure below).

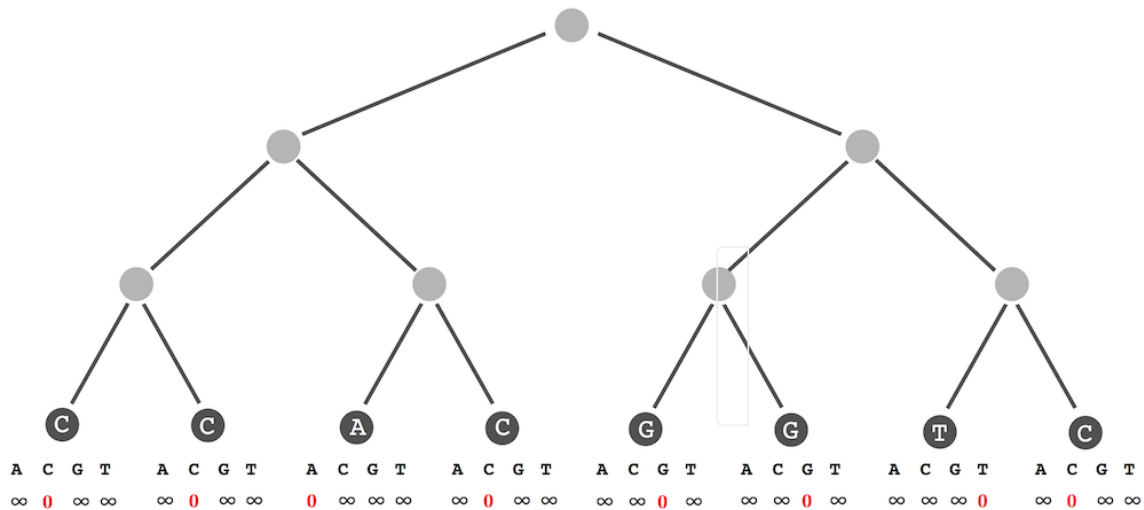


Figure: Initializing values $s_k(v)$ for all leaves v , represented above as an array for each leaf. We set $s_k(v)$ equal to zero if the leaf is labeled by symbol k ; otherwise, we set $s_k(v)$ equal to infinity.



Then v is connected to two “children” nodes (the nodes beneath v in T) that we arbitrarily denote as $Daughter(v)$ and $Son(v)$. The score $s_k(v)$ can be computed as the minimum of $s_i(Daughter(v)) + d_{i,k}$ over all possible symbols i , plus the minimum of $s_j(Son(v)) + d_{j,k}$ over all possible symbols j :

$$s_k(v) = \min_{\text{all symbols } i} \{s_i(Daughter(v)) + \delta_{i,k}\} + \min_{\text{all symbols } j} \{s_j(Son(v)) + \delta_{j,k}\}$$





This equation allows us to compute all values $s_k(v)$ by working our way upward in T from the leaves to the root, denoted root (see figure below). The subtree of the root is the entire tree T , and so the minimum parsimony score is given by the smallest score $s_k(\text{root})$ over all symbols k ,

$$\min_{\text{all symbols } k} s_k(\text{root}).$$

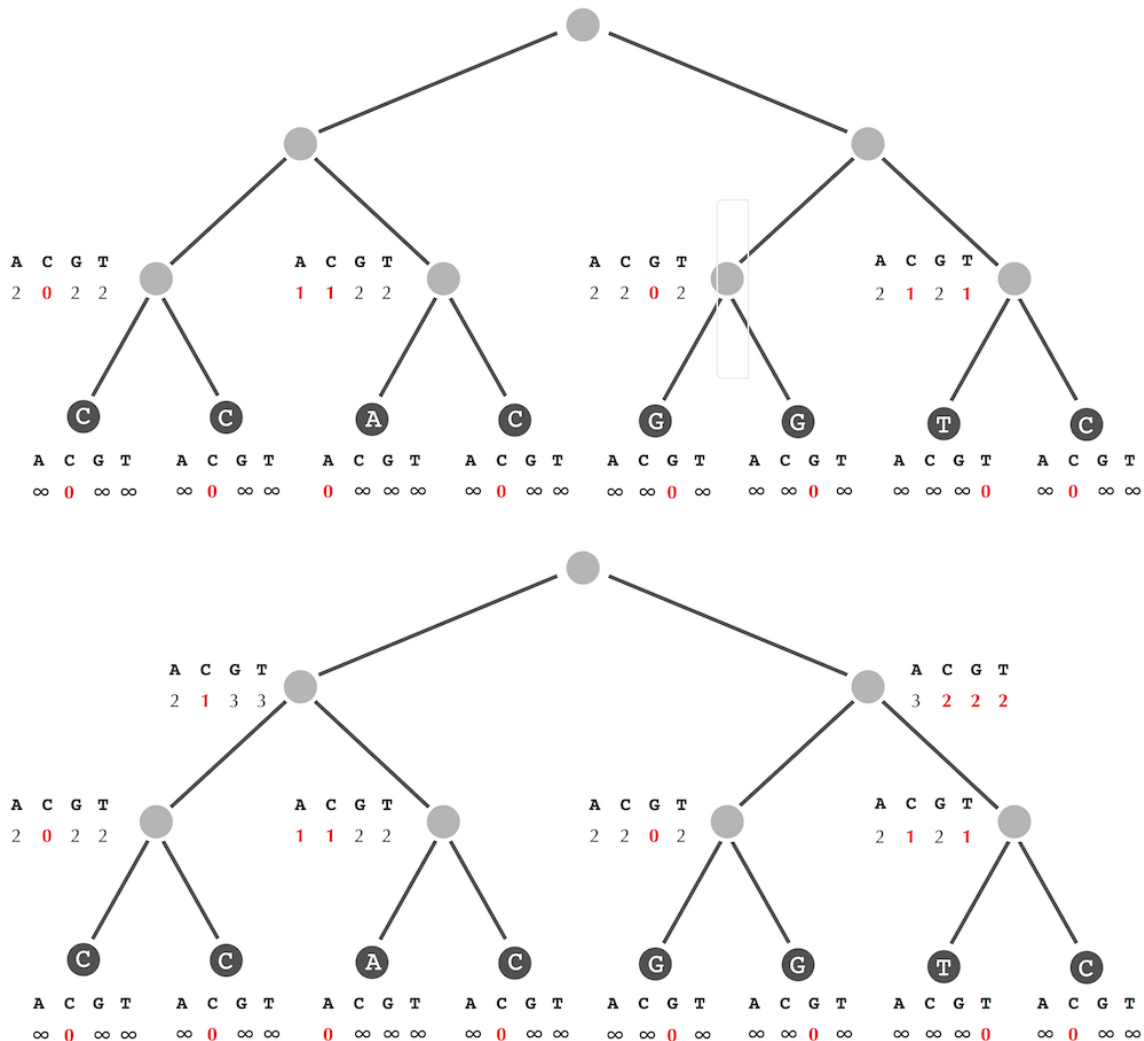
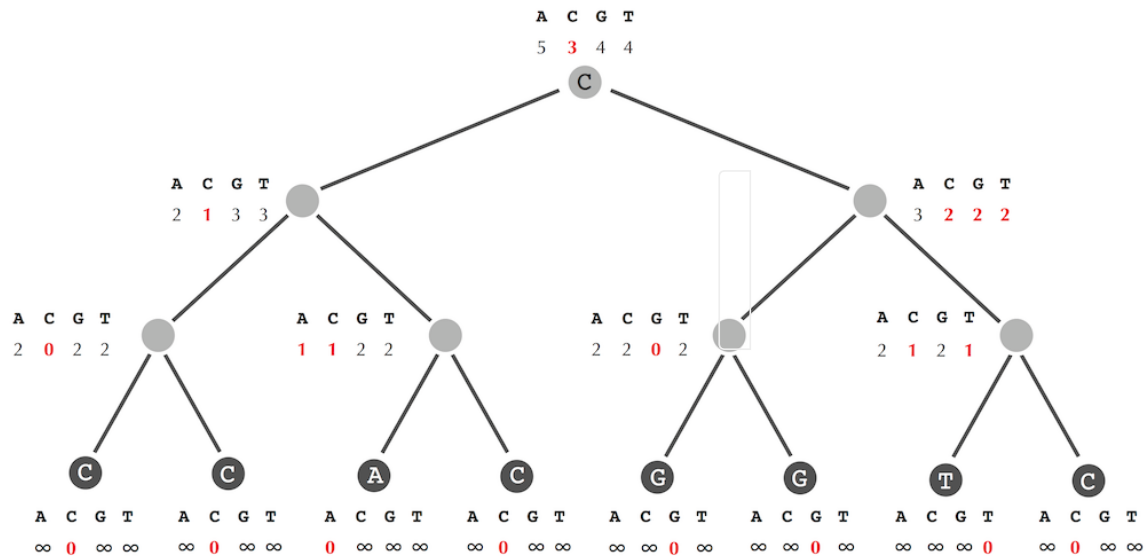


Figure: An illustration of **SmallParsimony** after initialization. The minimum value of $s_k(v)$ at each node v is shown in red. The parsimony score is equal to the minimum score at the root, which for this tree is equal to 3. This value corresponds to symbol C, and so when we begin backtracking to assign symbols to internal nodes, we assign nucleotide C to the root.



Once we compute the parsimony score of a tree T , we also need some way of assigning symbols to internal nodes of T . The minimum value of $s_k(\text{root})$ in the figure equal to 3, which is achieved when $k = C$. Assigning symbols to the remaining internal nodes is similar to the backtracking approach that we used for sequence alignment. Because you are already a pro at dynamic programming, we leave this task to you as an exercise.

Exercise Break: Assign nucleotides to all nodes of the tree below in order to solve the Small Parsimony Problem.





The pseudocode for **SmallParsimony** is shown below. It returns the parsimony score for a binary rooted tree T whose leaves are labeled by symbols from an array $Character$. At each iteration, it selects a node v and computes $s_k(v)$ for each symbol k in the alphabet. For each node v , **SmallParsimony** maintains a value $Tag(v)$, which indicates whether the node has been processed (i.e., $Tag(v) = 1$ if the array $s_k(v)$ has been computed and $Tag(v) = 0$ otherwise). We call an internal node of T **ripe** if its tag is 0 but its children's tags are both 1. **SmallParsimony** works upward from the leaves, finding a ripe node v at which to compute $s_k(v)$ at each step.

SmallParsimony(T , $Character$)

```

for each node  $v$  in tree  $T$ 
  if  $v$  is leaf
     $Tag(v) \leftarrow 1$ 
    for each symbol  $k$  in the alphabet
      if  $Character(v) = k$ 
         $s_k(v) \leftarrow 0$ 
      else
         $s_k(v) \leftarrow \infty$ 
  else
     $Tag(v) \leftarrow 0$ 
while there exist ripe nodes in  $T$ 
   $v \leftarrow$  a ripe node in  $T$ 
   $Tag(v) \leftarrow 1$ 
  for each symbol  $k$  in the alphabet
     $s_k(v) \leftarrow$  minimum over all symbols  $i$   $\{s_i(Daughter(v)) + \delta_{i,k}\} +$  minimum over all symbols  $j$   $\{s_j(Son(v)) + \delta_{j,k}\}$ 
return minimum over all symbols  $k$   $\{s_k(v)\}$ 

```

STOP and Think: What is the final ripe node processed by **SmallParsimony**, regardless of the order in which we process ripe nodes?



CODE CHALLENGE: Implement **SmallParsimony** to solve the Small Parsimony Problem.

Input: An integer n followed by an adjacency list for a rooted binary tree with n leaves labeled by DNA strings.

Output: The minimum parsimony score of this tree, followed by the adjacency list of the tree corresponding to labeling internal nodes by DNA strings in order to minimize the parsimony score of the tree.

Note: Remember to run **SmallParsimony** on each individual index of the strings at the leaves of the tree.

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradatasets/evolution/Small_Parsimony.txt)

Sample Input:

```
4
4->CAAATCCC
4->ATTGCGAC
5->CTGCGCTG
5->ATGGACGA
6->4
6->5
```

Sample Output:

```
16
ATTGCGAC->ATAGCCAC:2
ATAGACAA->ATAGCCAC:2
ATAGACAA->ATGGACTA:2
ATGGACGA->ATGGACTA:1
CTGCGCTG->ATGGACTA:4
ATGGACTA->CTGCGCTG:4
ATGGACTA->ATGGACGA:1
ATGGACTA->ATAGACAA:2
ATAGCCAC->CAAATCCC:5
ATAGCCAC->ATTGCGAC:2
ATAGCCAC->ATAGACAA:2
CAAATCCC->ATAGCCAC:5
```

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-10335/step/10



When the position of the root in the tree is unknown, we can simply assign the root to any edge that we like, apply **Small Parsimony** to the resulting rooted tree, and then remove the root. It can be shown that this method provides a solution to the following problem.

Small Parsimony Problem in an Unrooted Tree Problem: *Find the most parsimonious labeling of the internal nodes in an unrooted tree.*

Input: An unrooted binary tree with each leaf labeled by a string of length m .

Output: A labeling of all other nodes of the tree by strings of length m that minimizes the parsimony score of the tree.





CODE CHALLENGE: Solve the Small Parsimony in an Unrooted Tree Problem.

Input: An integer n followed by an adjacency list for an unrooted binary tree with n leaves labeled by DNA strings.

Output: The minimum parsimony score of this tree, followed by the adjacency list of the tree corresponding to labeling internal nodes by DNA strings in order to minimize the parsimony score of the tree.

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradatasets/evolution/Small_Parsimony_Unrooted_Tree.txt)

Sample Input:

```
4
TCGGCCAA->4
4->TCGGCCAA
CCTGGCTG->4
4->CCTGGCTG
CACAGGAT->5
5->CACAGGAT
TGAGTACC->5
5->TGAGTACC
4->5
5->4
```

Sample Output:

```
17
TCGGCCAA->CCAGGCAC:4
CCTGGCTG->CCAGGCAC:3
TGAGTACC->CAAGGAAC:4
CCAGGCAC->CCTGGCTG:3
CCAGGCAC->CAAGGAAC:2
CCAGGCAC->TCGGCCAA:4
CACAGGAT->CAAGGAAC:4
CAAGGAAC->CACAGGAT:4
CAAGGAAC->TGAGTACC:4
CAAGGAAC->CCAGGCAC:2
```

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-10335/step/12



SmallParsimony does not help us if we do not know the evolutionary tree in advance. In this case, we must find a tree as well as assign ancestral strings to all internal nodes of this tree in order to minimize the parsimony score. The figure below shows the solution of the Small Parsimony in an Unrooted Tree Problem for one tree with four leaves, where the leaves are assigned the strings from the toy mammal alignment, reproduced below.

Chimp	ACGTAGGCCT
Human	ATGTAAGACT
Seal	TCGAGAGCAC
Whale	TCGAAAGCAT

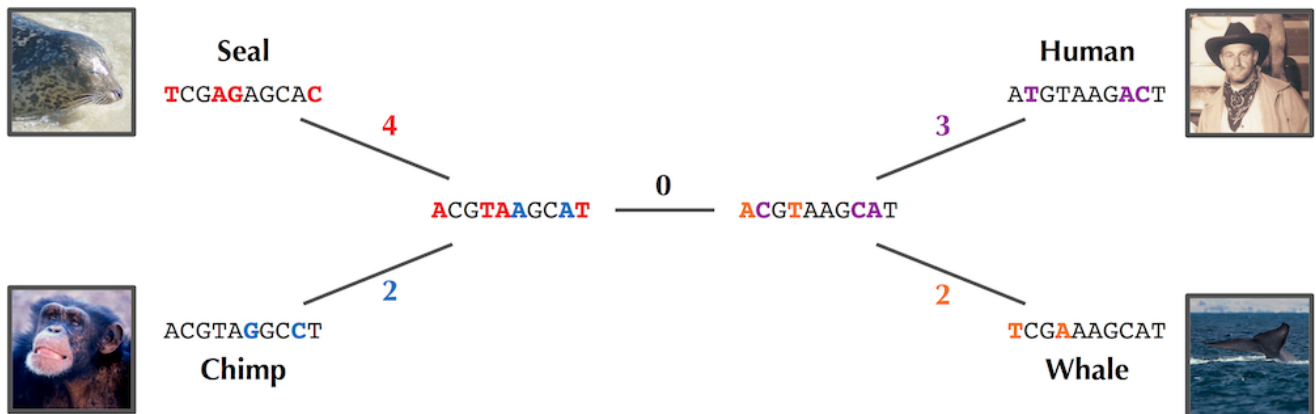


Figure: The three unrooted binary tree structures for the four species in the toy alignment, with internal nodes labeled by a solution of the Small Parsimony in an Unrooted Tree Problem. The first tree solves the Large Parsimony Problem because it has smaller parsimony score (8) than the other two trees (11 each).



Large Parsimony Problem: Given a set of strings, find a tree (with leaves labeled by all these strings) having minimum parsimony score.

Input: A collection of strings of equal length.

Output: An unrooted binary tree T that minimizes the parsimony score among all possible unrooted binary trees with leaves labeled by these strings.

Unfortunately, it turns out that the Large Parsimony problem is *NP*-Complete, in part because the number of different trees grows very quickly with respect to the number of leaves. As a workaround, we will use a greedy heuristic that explores some but not all trees. First, note that in any unrooted binary tree, the removal of an internal edge, along with the two internal nodes that this edge connects, results in four subtrees, which we will call W , X , Y , and Z (see figure on the previous step). These four subtrees can be combined into a tree in three different ways, which we denote $WX|YZ$, $WY|XZ$, and $WZ|XY$. These three trees are called **nearest neighbors**; a **nearest neighbor interchange** operation replaces a tree with one of its nearest neighbors.

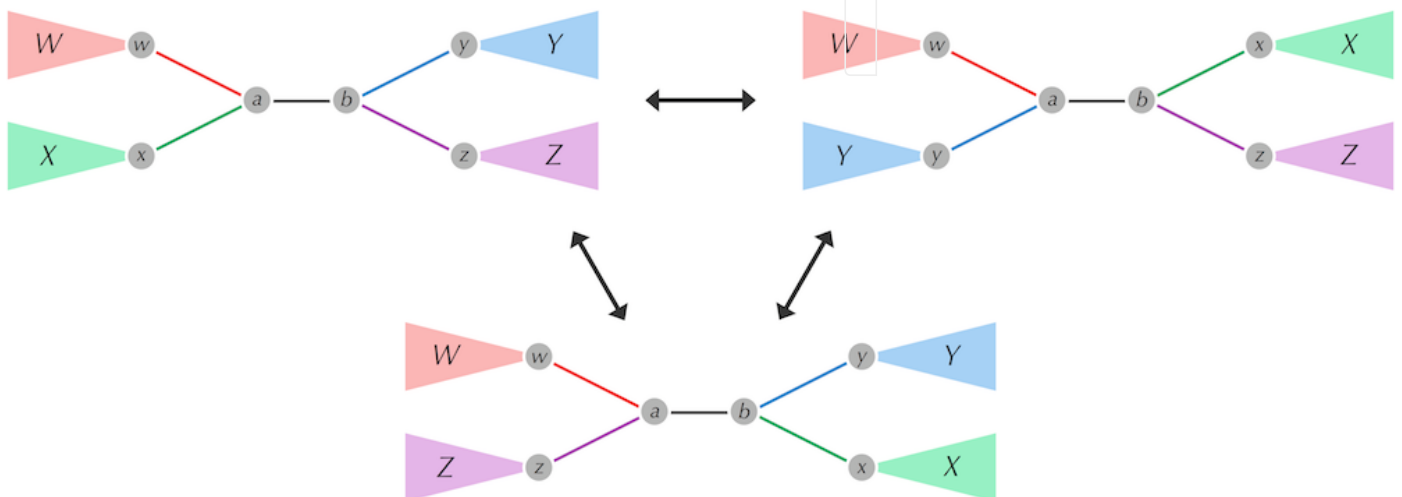


Figure: A nearest neighbor interchange on the internal edge (a,b) , shown in black, results from rearranging the four colored subtrees W , X , Y , and Z , which are rooted at w , x , y , and z , respectively. The nearest neighbor interchange operation requires us to break one edge connected to a and another edge connected to b , then rejoin the edges in a new way. The three possible tree structures resulting from nearest neighbor interchanges on (a, b) can be represented as $WX|YZ$ (top left), $WY|XZ$ (top right), and $WZ|XY$ (bottom).

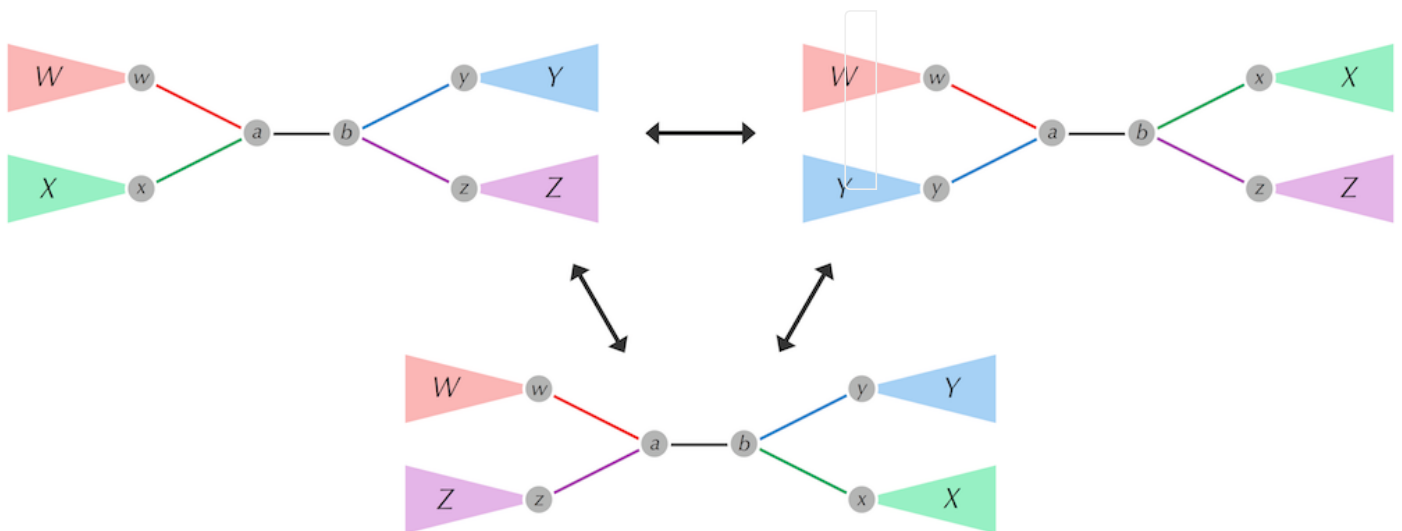


Nearest Neighbors of a Tree Problem: *Given an edge in a binary tree, generate the two neighbors of this tree.*

Input: An internal edge in a binary tree.

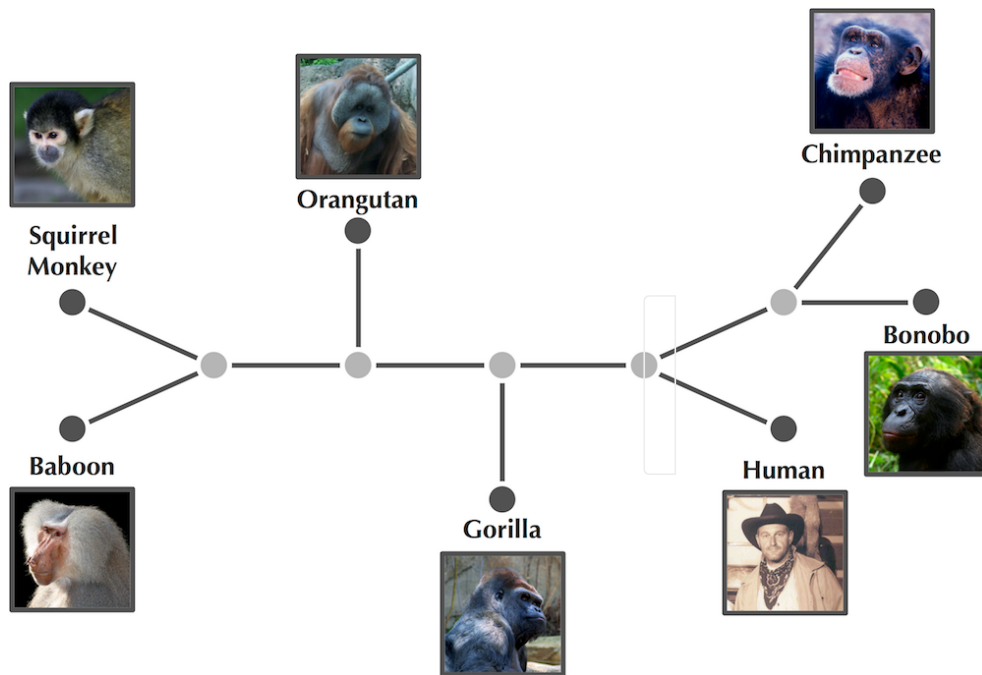
Output: The two nearest neighbors of this tree (for the given internal edge).

Like the 2-break operation in Chapter 6, a nearest neighbor interchange corresponds to replacing two edges in the tree by two new edges. For example, denote the internal edge of a nearest neighbor interchange as (a, b) ; denote the remaining nodes adjacent to a as w and x ; and denote the remaining nodes adjacent to b as y and z . The tree on the right in the figure below (reproduced from the previous step) is obtained from the tree on the left by removing edges (a, x) and (b, y) and substituting them with (a, y) and (b, x) . The tree on the bottom is obtained from the tree on the left by removing edges (a, x) and (b, z) and substituting them with (a, z) and (b, x) .





Exercise Break: Find the two nearest neighbors of the tree in the figure below for the internal edge connecting the parents of gorilla and human.





Exercise Break: The figure below shows all possible unrooted binary trees with five leaves. Find a pair of these trees that are the “farthest apart” in that they require the maximum number of nearest neighbor interchanges to transform one tree into the other.

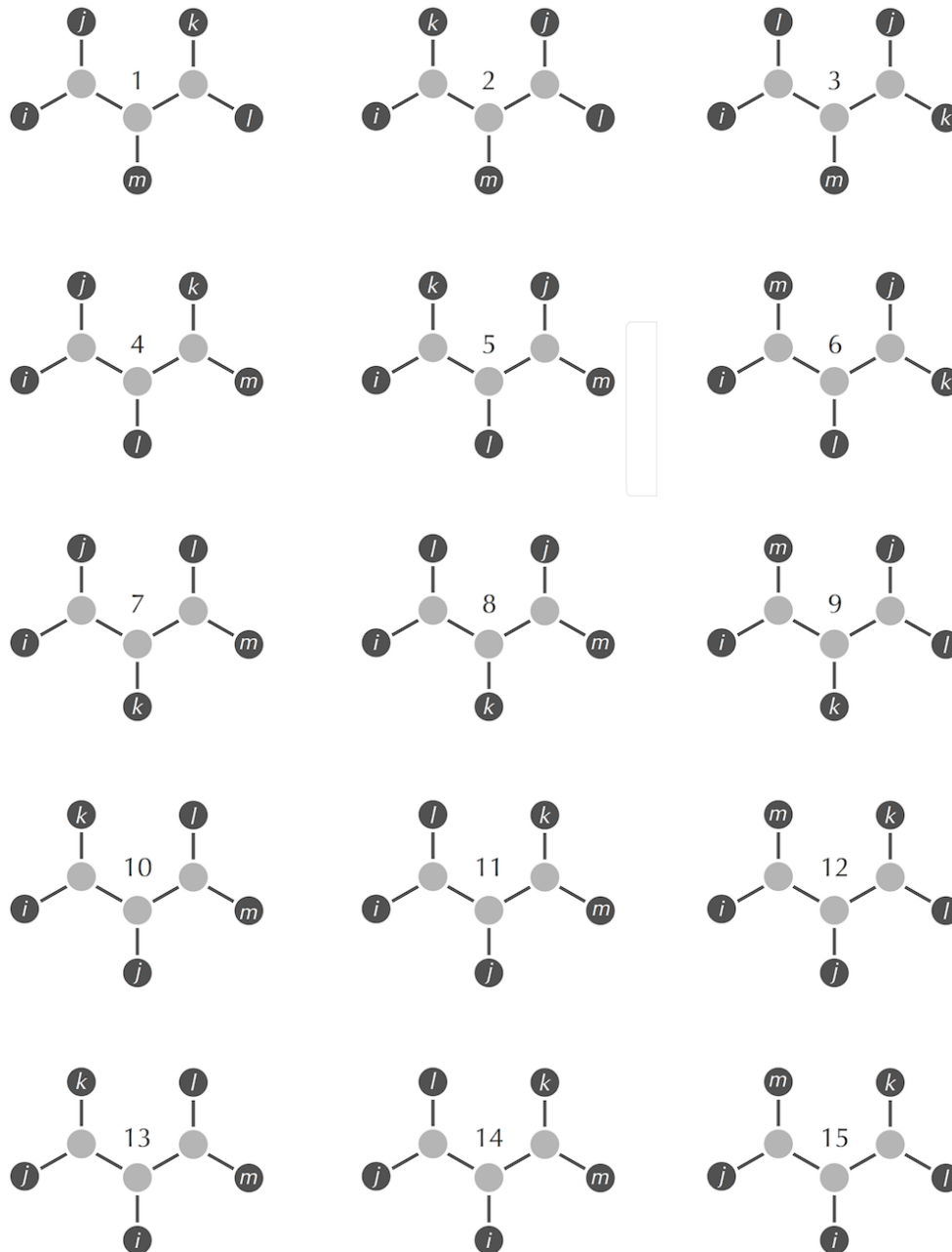


Figure: All fifteen unrooted binary trees with five labeled leaves. Tree 1 can be transformed into trees 4, 7, 12, and 15 by a single nearest neighbor interchange. Note that every tree has the same structure; this is not the case for trees containing more than five leaves.



CODE CHALLENGE: Solve the Nearest Neighbors of a Tree Problem.

Input: Two internal nodes a and b specifying an edge e , followed by an adjacency list of an unrooted binary tree.

Output: Two adjacency lists representing the nearest neighbors of the tree with respect to e . Separate the adjacency lists with a blank line.

Extra Dataset http://bioinformaticsalgorithms.com/data/extradata/evolution/Nearest_Neighbors.txt

Sample Input:

```
5 4
0->4
4->0
1->4
4->1
2->5
5->2
3->5
5->3
4->5
5->4
```

Sample Output:

```
1->4
0->5
3->4
2->5
5->2
5->4
5->0
4->1
4->5
4->3

1->5
0->4
3->4
2->5
5->2
5->4
5->1
4->0
4->5
4->3
```

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-10336/step/6



The **nearest neighbor interchange** heuristic for the Large Parsimony Problem starts from an arbitrary unrooted binary tree. It assigns input strings to arbitrary leaves of this tree, assigns strings to the internal nodes of the tree by solving the Small Parsimony Problem in an Unrooted Tree, and then moves to the nearest neighbor that provides the best improvement in the parsimony score. At each iteration, the algorithm explores all internal edges of a tree and generates all nearest neighbor interchanges for each internal edge. If a nearest neighbor with smaller parsimony score is found, then the algorithm selects the one with smaller parsimony score (ties are broken arbitrarily) and iterates again; otherwise, the algorithm terminates.





CODE CHALLENGE: Implement the nearest neighbor interchange heuristic for the Large Parsimony Problem.

Input: An integer n , followed by an adjacency list for an unrooted binary tree whose n leaves are labeled by DNA strings and whose internal nodes are labeled by integers.

Output: The parsimony score and unrooted labeled tree obtained after every step of the nearest neighbor interchange heuristic. Each step should be separated by a blank line.

Note: Depending on how your code breaks ties, you may obtain a different solution than the one we provide. As a result, the parsimony score at each step may vary.

Extra Dataset http://bioinformaticsalgorithms.com/data/extradatasets/evolution/Large_Parsimony_Heuristic_with_NNI.txt

Sample Input:

```
5
GCAGGGTA->5
TTTACGCG->5
CGACCTGA->6
GATTCCAC->6
5->TTTACGCG
5->GCAGGGTA
5->7
TCCGTAGT->7
7->5
7->6
7->TCCGTAGT
6->GATTCCAC
6->CGACCTGA
6->7
```

Sample Output:

22

TCCGTAGT->TCAGCGGA:4
GATTCCAC->GAACCCGA:4
CGACCTGA->GAACCCGA:3
TTTACGCG->TCAGCGGA:5
TCAGCGGA->TTTACGCG:5
TCAGCGGA->GCAGCGGA:1
TCAGCGGA->TCCGTAGT:4
GCAGGGTA->GCAGCGGA:2
GCAGCGGA->GAACCCGA:3
GCAGCGGA->GCAGGGTA:2
GCAGCGGA->TCAGCGGA:1
GAACCCGA->GATTCCAC:4
GAACCCGA->CGACCTGA:3
GAACCCGA->GCAGCGGA:3

21

TCCGTAGT->TCTGCGGA:4
GATTCCAC->GCTGCGGA:5
CGACCTGA->GCAGCGGA:4
TTTACGCG->TCTGCGGA:4
TCTGCGGA->TTTACGCG:4
TCTGCGGA->GCTGCGGA:1
TCTGCGGA->TCCGTAGT:4
GCAGGGTA->GCAGCGGA:2
GCTGCGGA->GCAGCGGA:1
GCTGCGGA->GATTCCAC:5
GCTGCGGA->TCTGCGGA:1
GCAGCGGA->CGACCTGA:4
GCAGCGGA->GCAGGGTA:2
GCAGCGGA->GCTGCGGA:1

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-10336/step/8



Exercise Break: A later chapter will describe how the alcohol dehydrogenase (Adh) gene helps yeast to produce alcohol. To study the evolution of this gene, biologists constructed a multiple alignment of the Adh genes from various yeast species. Use the following alignment to reconstruct the evolutionary tree of various yeast species and an ancient yeast Adh ancestor gene.

[Download Alignment \(http://bioinformaticsalgorithms.com/data/realdatasets/Evolution/yeast_Ahd_alignment.txt\)](http://bioinformaticsalgorithms.com/data/realdatasets/Evolution/yeast_Ahd_alignment.txt)





This chapter has provided us with a number of algorithms for constructing evolutionary trees from genetic data, but this does not mean that phylogenetic analysis is easy. For example, the identity of the chimpanzee's closest relative remained undecided until the mid-1990s (see figure below), and the question of whether mice are closer to humans than to dogs is still the subject of debate (see figure on the next step).

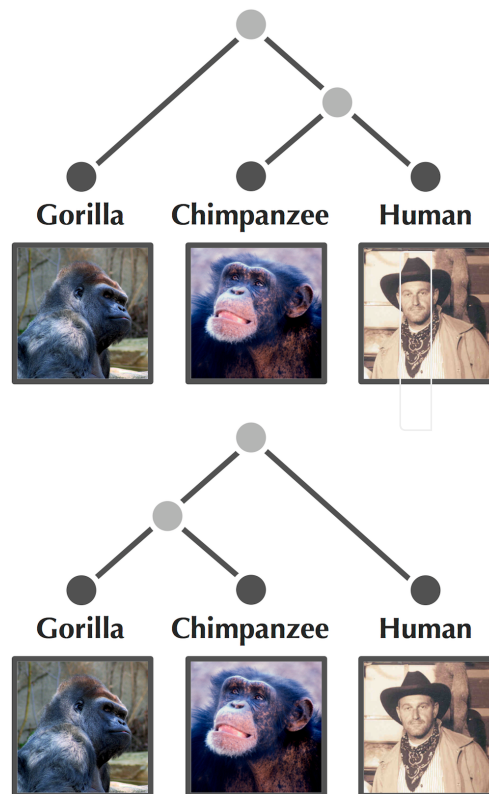


Figure: (Top) Analysis of beta-globin genes in human, chimpanzee, and gorilla suggests a human-chimpanzee split. (Bottom) Analysis of dopamine D4 receptor gene suggests a gorilla-chimpanzee split. Image courtesies: Frans de Waal (chimp), Kabir Bakie (gorilla).

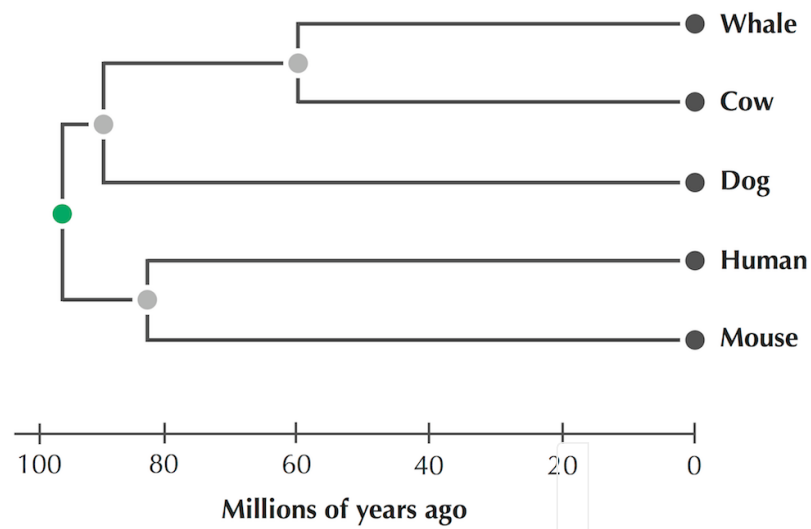


Figure: Just fifteen years ago, biologists believed that dogs are evolutionarily closer to humans than mice are. However, recent studies suggest otherwise, as shown in the above phylogeny.



Janice Trahan met Dr. Richard Schmidt in 1982 when she began working as a nurse in Lafayette, Louisiana. Both Janice and Richard were married and had children, but they fell in love. Janice soon divorced her husband, but although Richard promised that he would divorce his wife, he never did. After twelve years, she got tired of waiting, and broke off the relationship. Two weeks later, she awoke in the middle of the night to see Richard standing over her, a syringe in his hand.

Although Janice had broken up with Richard, she was not surprised to see him. She had even left her door unlocked for him because he had been giving her vitamin B-12 injections for her chronic fatigue. Since Janice had B-12 injections before, she knew what to expect. This time, however, she experienced scorching pain as Richard squeezed the syringe.





A few months later, Janice tested positive for HIV, and she accused Richard of infecting her via an injection. The police detective in Lafayette had never heard anything as bizarre as this revenge story - a syringe tainted with HIV had never been used as a murder weapon. At first, he suspected that Janice had fabricated the story to tarnish the reputation of her former lover. Nevertheless, he started an investigation by collecting samples from several HIV patients in Lafayette.

After investigating hospital records, the detective found that Richard had taken blood from Donald McClelland, an HIV patient, on the same day that he had injected Janice. Now the forensic challenge was to determine whether the HIV strain taken from McClelland was similar to Janice's strain, a difficult task because HIV evolves quickly. After HIV DNA was collected from the victim, McClelland, and many other unrelated HIV-infected patients from Lafayette, scientists constructed the evolutionary tree of these HIV viruses and found that the viruses sampled from Janice and McClelland formed a subtree of this tree (see figure below).

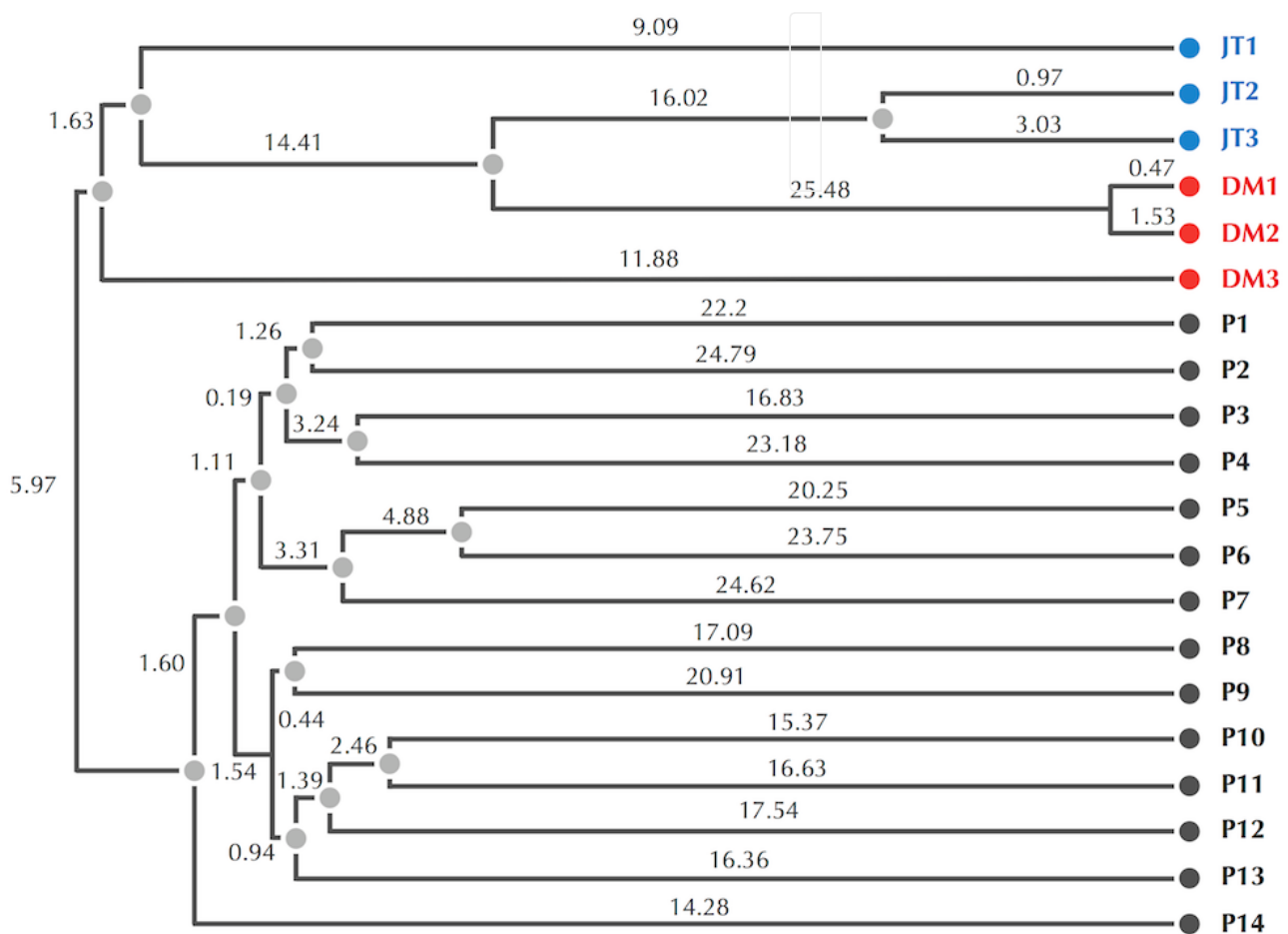


Figure: An evolutionary tree of HIV viruses taken from various patients in Lafayette. Samples from Janice Trahan (blue leaves JT1, JT2, and JT3) and Donald McClelland (red leaves DM1, DM2, and DM3) are clustered together and are rather different than sequences from other patients from Lafayette (labeled P1 to P14).



The case “State of Louisiana vs. Richard Schmidt” went to trial in 1998. The prominent evolutionary biologist David Hillis presented the evolutionary tree as evidence of the crime, demonstrating that Janice’s sequence had been derived (with some variations) from McClelland’s sequence. Richard Schmidt was then sentenced to 50 years in prison for attempted murder.

STOP and Think: If you had been Richard Schmidt’s attorney, how would you have argued for his innocence?

Challenge Problem: Construct the evolutionary tree for a different HIV protein. Does this tree also support conviction of Dr. Schmidt? Reconstruct the ancestral HIV sequences at the internal nodes of the resulting trees.

Download Dataset [_ \(http://bioinformaticsalgorithms.com/data/challengedatasets/lafayette_SARS_RT.txt\)](http://bioinformaticsalgorithms.com/data/challengedatasets/lafayette_SARS_RT.txt)





For many years, biologists could not agree on whether the giant panda should be classified as a bear or as a raccoon. Although giant pandas look like bears, they have features that are unusual for bears and typical of raccoons: they do not hibernate in the winter, and their male genitalia are tiny and backward-pointing. As a result, Edwin Colbert wrote in 1938:

So the quest has stood for many years with the bear proponents and the raccoon adherents and the middle-of-the-road group advancing their several arguments with the clearest of logic, while in the meantime the giant panda lives serenely in the mountains of Szechuan with never a thought about the zoological controversies he is causing by just being himself.

Whereas analysis of anatomical and behavior characters (such as hibernation versus non-hibernation) only led to unsettled debates, the analysis of genetic characters by Stephen O'Brien in 1985 demonstrated that giant pandas are indeed more closely related to bears than raccoons (see figure below).

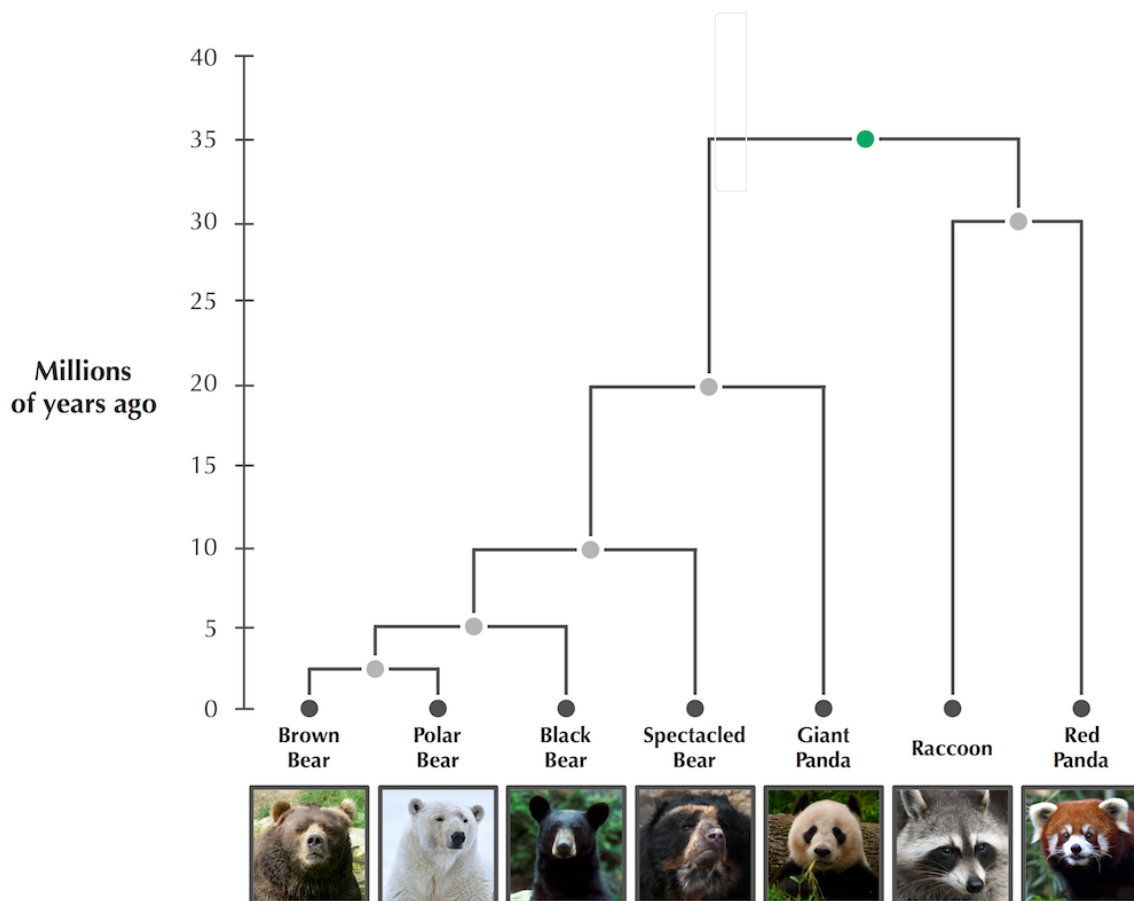


Figure: An evolutionary tree of bears and raccoons. Image courtesies: Alan D. Wilson (polar bear), Mark Dumont (spectacled bear), Peter Meenen (red panda), Wikimedia commons user Darkone (raccoon).



In 1987, Rebecca Cann, Mark Stoneking and Allan Wilson constructed an evolutionary tree of **mitochondrial DNA (mtDNA)** of 133 people representing African, Asian, Australian, Caucasian, and New Guinean ethnic groups. This tree led to the **Out of Africa** hypothesis, which claims that humans have a common ancestor who lived in Africa. This study turned the question of human origins into an algorithmic problem.

The mtDNA evolutionary tree showed a trunk splitting into two major branches (see figure below). One branch, containing the bottom five individuals in the figure below, consisted only of Africans, whereas the other branch included some modern Africans as well as all people belonging to other ethnic groups.

STOP and Think: After looking at the tree in the figure below, where do you think all humans came from?



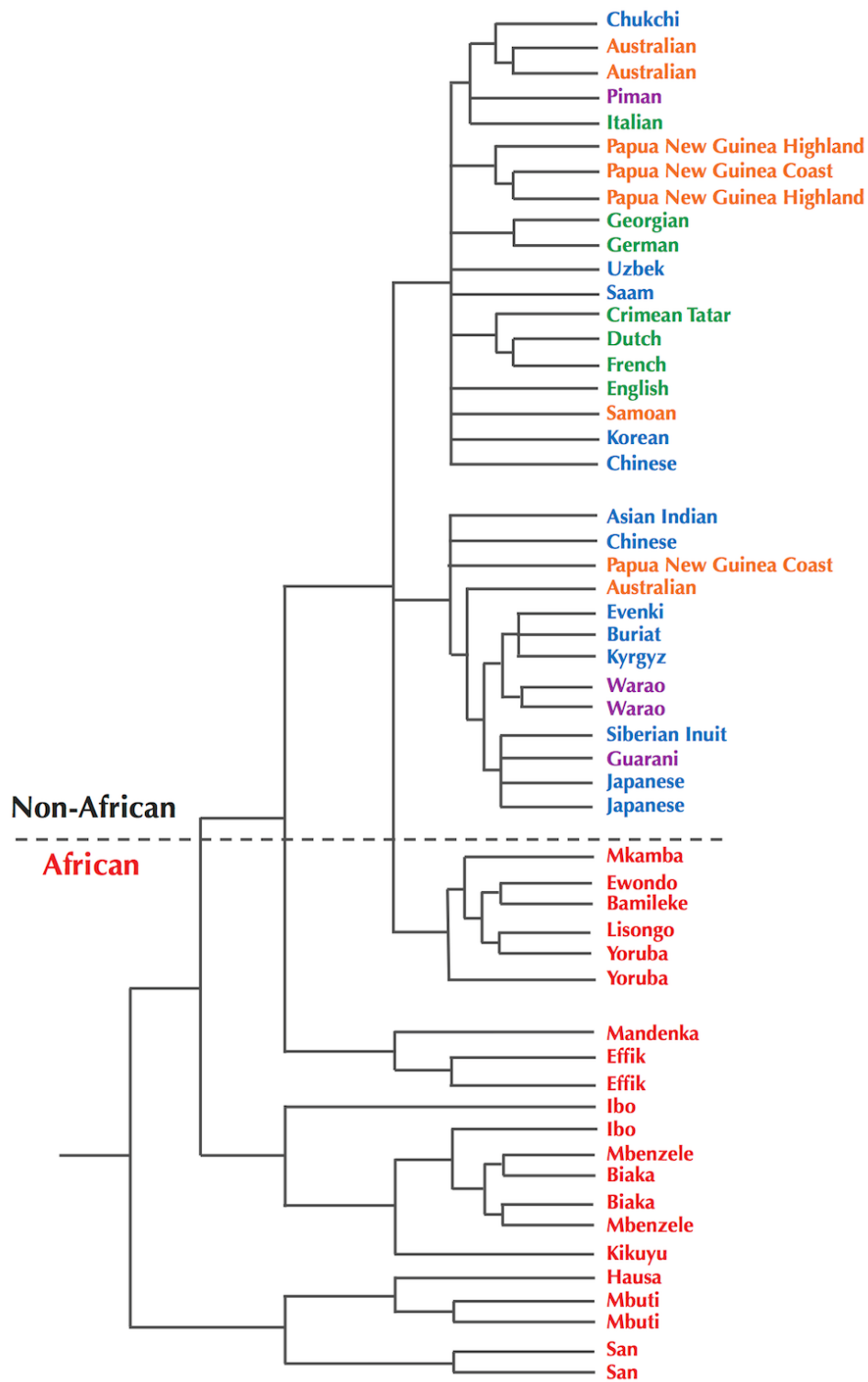


Figure: Evolutionary tree constructed for various human mitochondrial genomes from Africa (red), Asia (blue), North and South America (purple), Europe (green), and Oceania (orange). “PNG” denotes “Papua New Guinea”. The dashed line divides African from non-African genomes.



If humans populated Africa before Asia, then African genomes started to diverge from each other earlier than Asian genomes. Thus, we would expect to find that African genomes, which had more time to diverge from each other, have more mutations (compared to each other and other genomes) than Asian genomes. This reasoning gives us a hint on how to check whether the human race spread from Africa.

African genomes are indeed more diverse than genomes from other continents, which led Wilson and colleagues to conclude that the African lineage is the oldest and that modern humans trace their roots back to Africa. Thus, a population of Africans, the first modern humans, forms one subtree, whereas another subtree represents a subgroup that left Africa and later spread out to the rest of the world. Using the mitochondrial tree, Wilson and colleagues further estimated that humans emerged from Africa 130,000 years ago, with racial differences arising only 50,000 years ago. The figure below shows putative human migration patterns derived from genomic data.

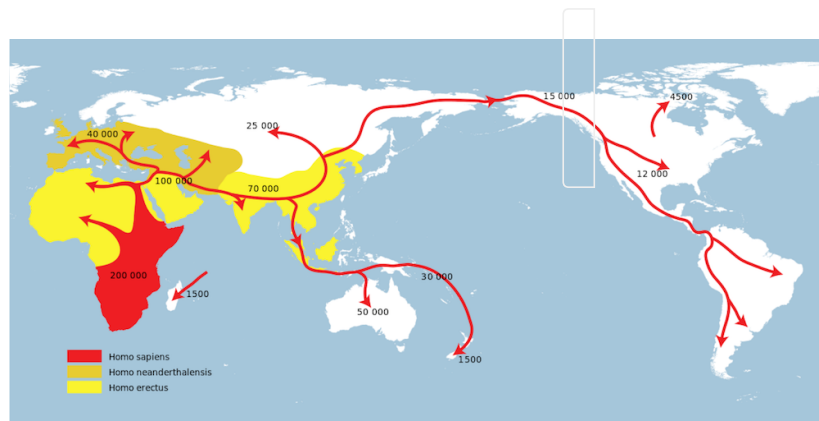


Figure: Putative routes of human migration derived from genomic data labeled by the number of years ago these migrations occurred.